

MODSERVE: Scalable and Resource-Efficient Large Multimodal Model Serving

Authors: Qiu, Mohan, Goiri, Bansal, Biswas, Khare, Zhang,
Ramjee, Zhao, Choukse, Shen
Fonseca Microsoft · University of Virginia

Presenters: Himnish Hunma, Enya Sun

Agenda

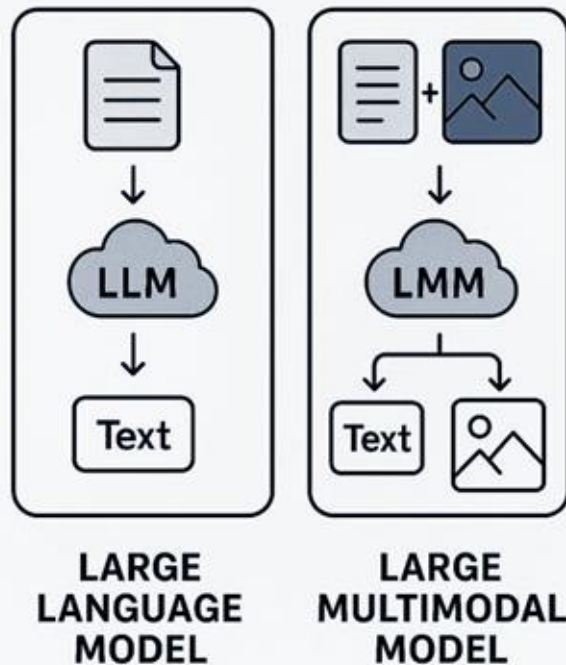
- Motivation
- Background
- LMM Characterization / Analysis
- System Design
- Evaluation
- Strengths & Weaknesses
- Limitations and Future Work

Motivation

Efficient LMM serving is challenging

- LMMs have complex architectures and multi-stage inference pipelines
- Monolithic systems don't scale effectively while meeting SLOs

THE DIFFERENCE BETWEEN A LLM AND A LMM

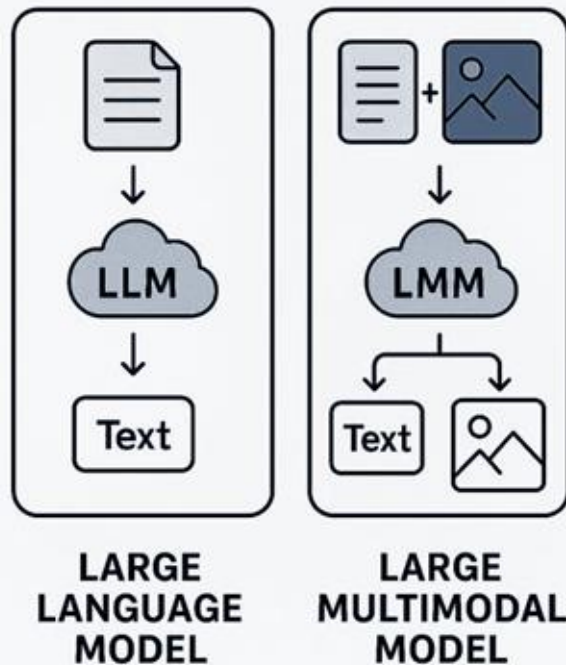


Motivation

Efficient LMM serving is challenging

- LMMs have complex architectures and multi-stage inference pipelines
- Monolithic systems don't scale effectively while meeting SLOs
- This paper contributes:
 - A **system characterization** on LMM serving

THE DIFFERENCE BETWEEN A LLM AND A LMM

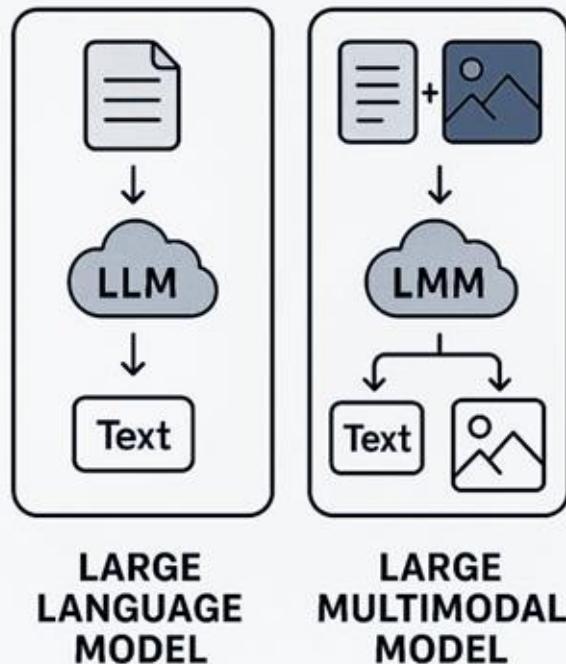


Motivation

Efficient LMM serving is challenging

- LMMs have complex architectures and multi-stage inference pipelines
- Monolithic systems don't scale effectively while meeting SLOs
- This paper contributes:
 - A **system characterization** on LMM serving
 - A large open-source dataset containing sampled

THE DIFFERENCE BETWEEN A LLM AND A LMM

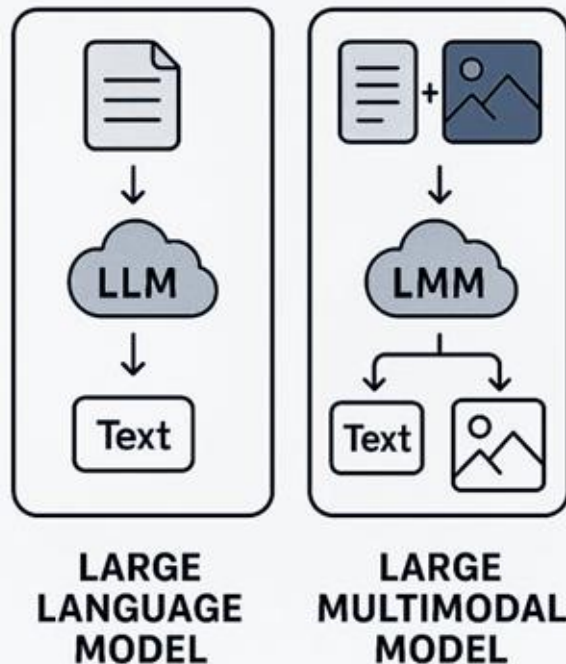


Motivation

Efficient LMM serving is challenging

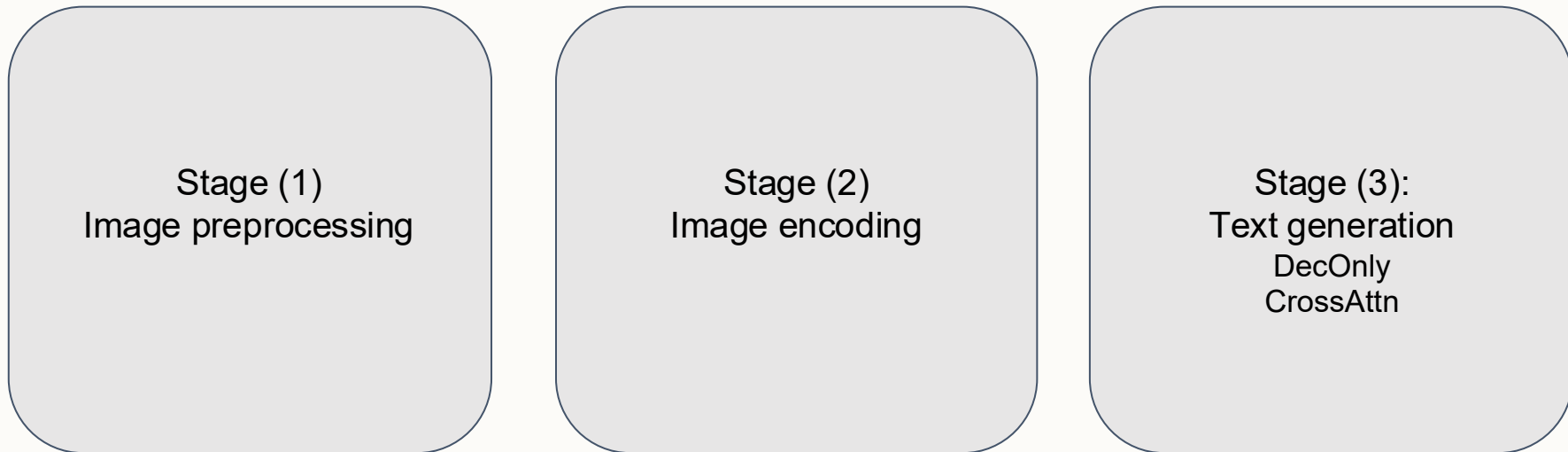
- LMMs have complex architectures and multi-stage inference pipelines
- Monolithic systems don't scale effectively while meeting SLOs
- This paper contributes:
 - A **system characterization** on LMM serving
 - A large open-source dataset containing sampled production azure LMM inference traces
 - The design, implementation, and evaluation of **ModServe**

THE DIFFERENCE BETWEEN A LLM AND A LMM



Background

Typical pipeline of LMM inference in visual-understanding tasks



(Stages 1-2) Image preprocessing and encoding

Image tiling procedure

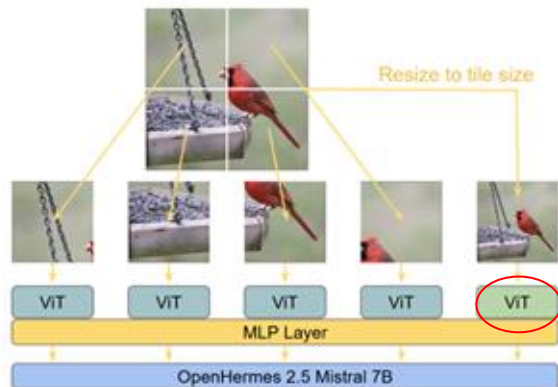
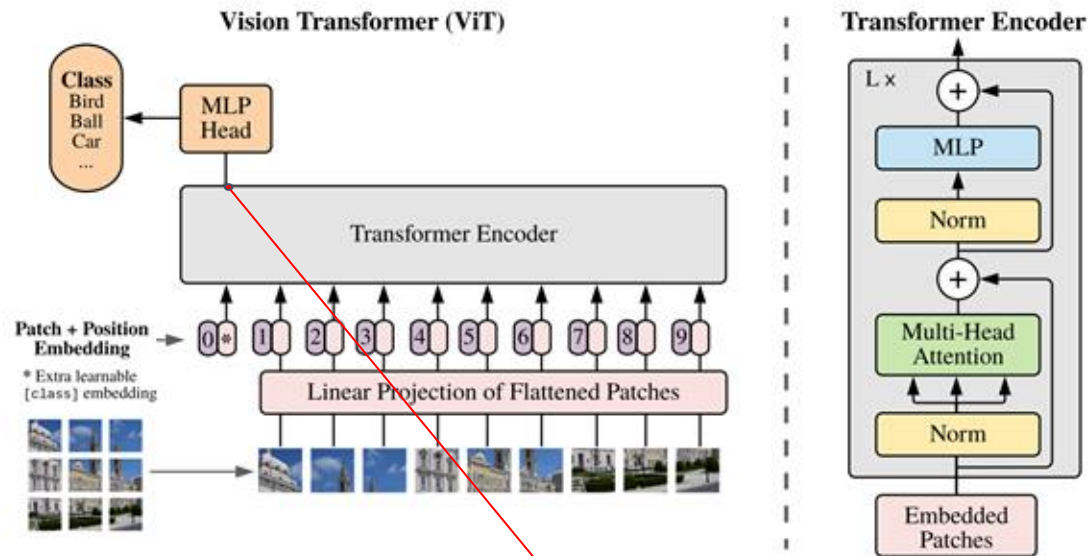
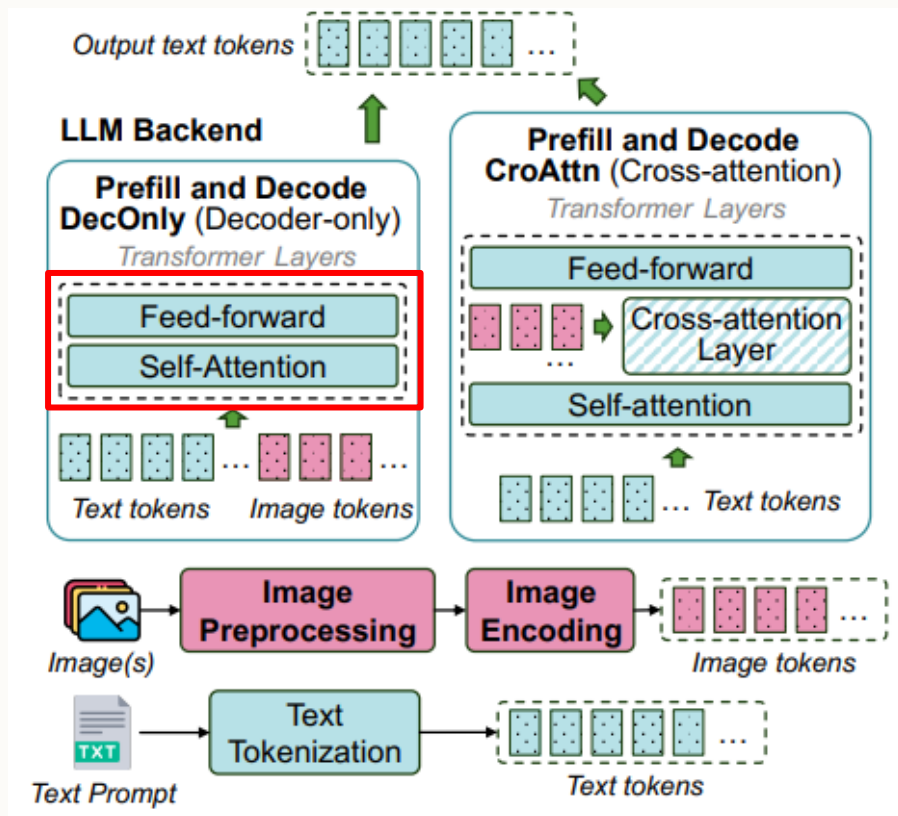


Figure 1: The image splitting approach processes high-resolution images through multiple vision encoder towers, with optional global context.

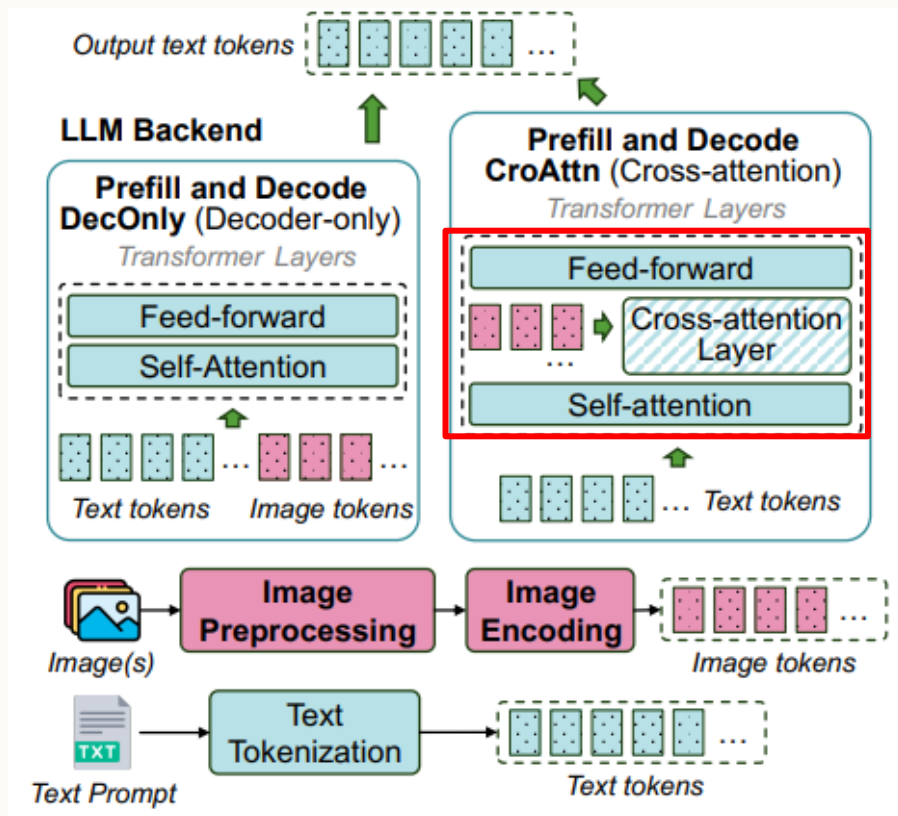


Grab
embeddings
from here

(Stages 3) Text Generation



(Stages 3) Text Generation



Analysis - LMM Characterization

Insights for optimizing LMM inference

Decoupled execution

Tackle diverse performance characteristics and resource +
model configs in different stages and

Encoder parallelization

Reduce latency and HoL blocking

Modality-aware routing

Manage bursts and mitigate tail latency spikes

Insights 1 - 5

Characterization on Open-Source LMMs

Per-stage Latency Breakdown

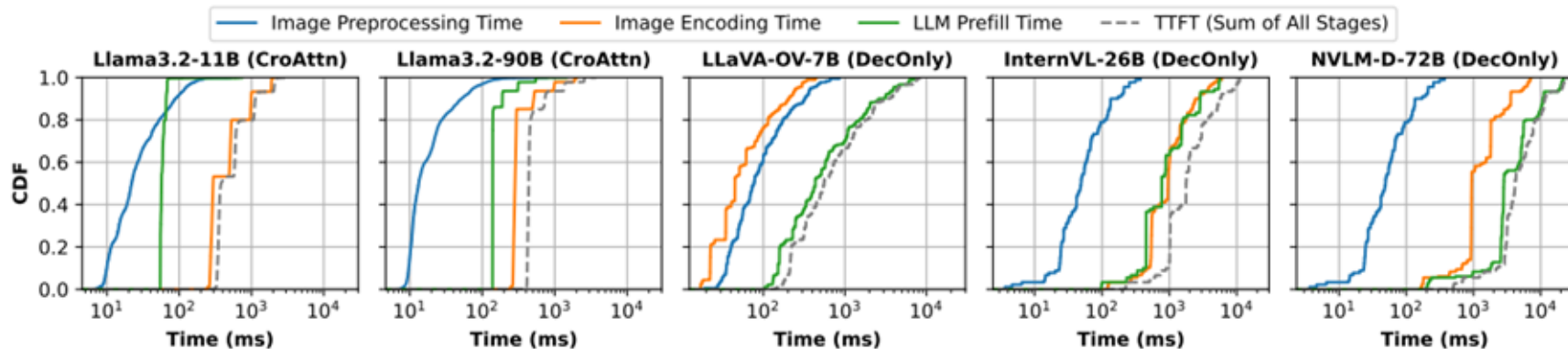


Figure 5. Per-stage request latency breakdown analysis across representative open-source LMMs deployed using default tensor parallelism (TP) as described in Table 1. TTFT (dashed line) is the sum of the latency from each inference stage.

Insight 1: A major portion of the TTFT is spent on image encoding, particularly for CroAttn models, making image encoding optimization critical to meet TTFT SLOs

Compute Characteristics of LMM Stages

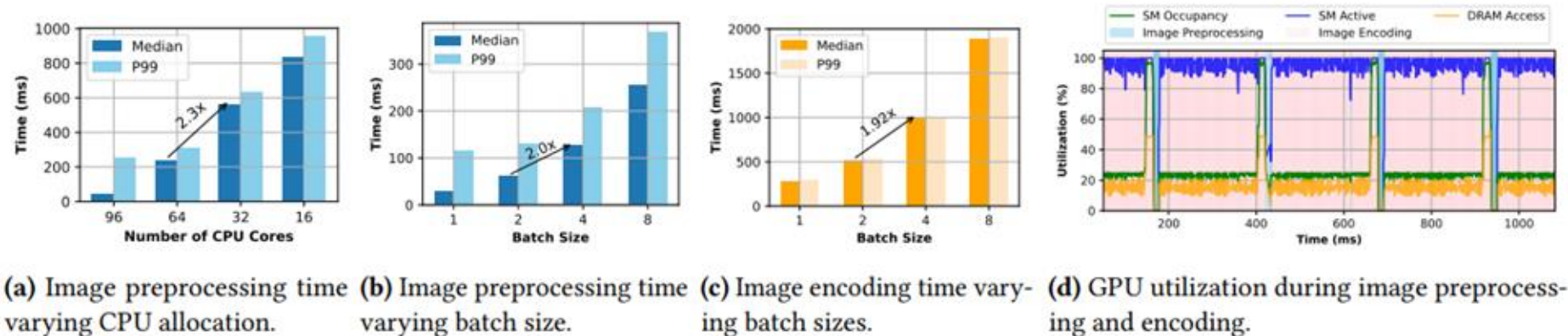


Figure 6. Compute characteristics of image preprocessing and encoding. Both stages are compute-bound.

Insight 2: Image preprocessing and encoding are both compute-intensive similar to LLM prefill stage. The independence of image computations in a multimodal request enables parallelization of image preprocessing and encoding across multiple instances

Latency-Accuracy Profiles across LMMs.

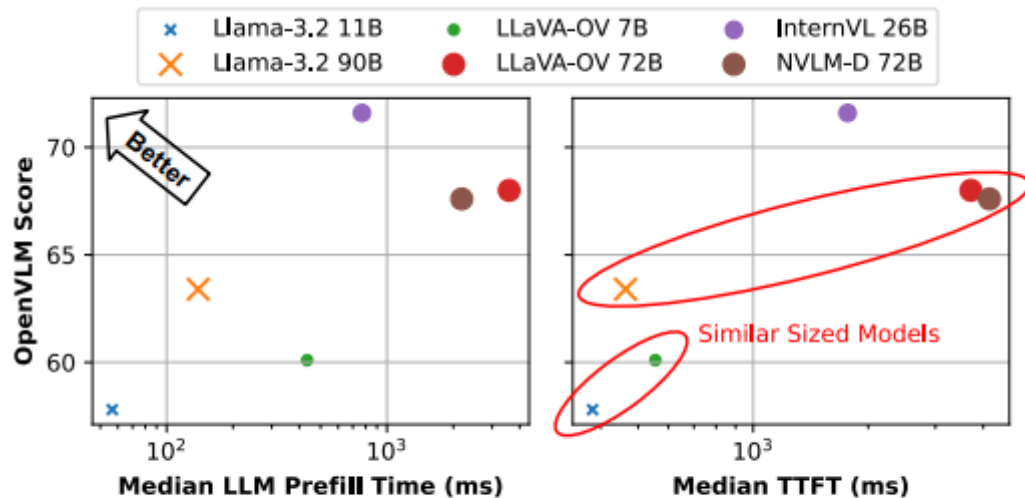


Figure 7. LMM accuracy vs. prefill/TTFT efficiency.

Insight 3: DecOnly models exhibit 10× worse prefill latency than similar-sized CroAttn models, leading to less TTFT SLO headroom for the image encoding and thus necessitating higher scalability for image workloads.

Latency-Accuracy Profiles across LMMs.

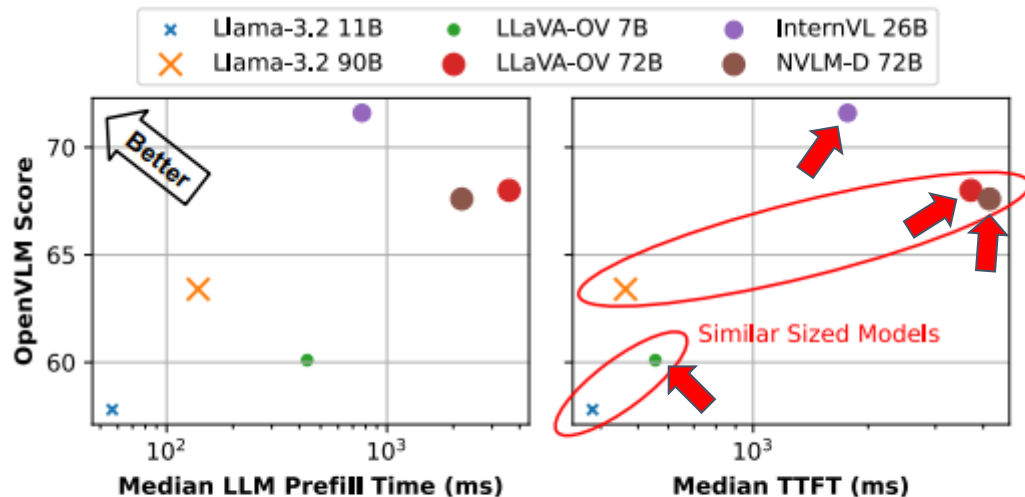


Figure 7. LMM accuracy vs. prefill/TTFT efficiency.

Question: Why do DecOnly models exhibit such worse prefill latency?

Insight 3: DecOnly models exhibit 10× worse prefill latency than similar-sized CroAttn models, leading to less TTFT SLO headroom for the image encoding and thus necessitating higher scalability for image workloads.

Impact of Batching

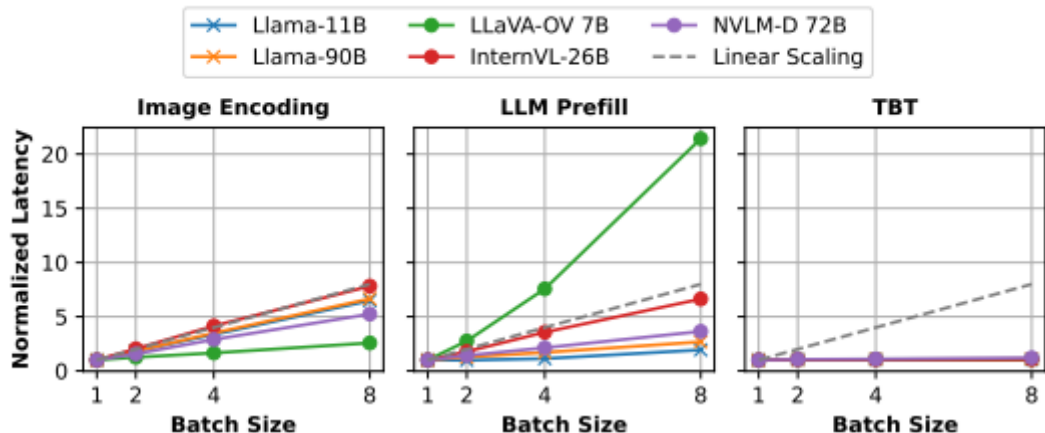


Figure 8. Median latency vs. batch size per LMM stage on GPUs. Latency is normalized to that at batch size one.

Insight 4: The effectiveness of batching varies for each LMM component and is model-specific. LMM request batching should thus be tailored to each stage.

Impact of Parallelism

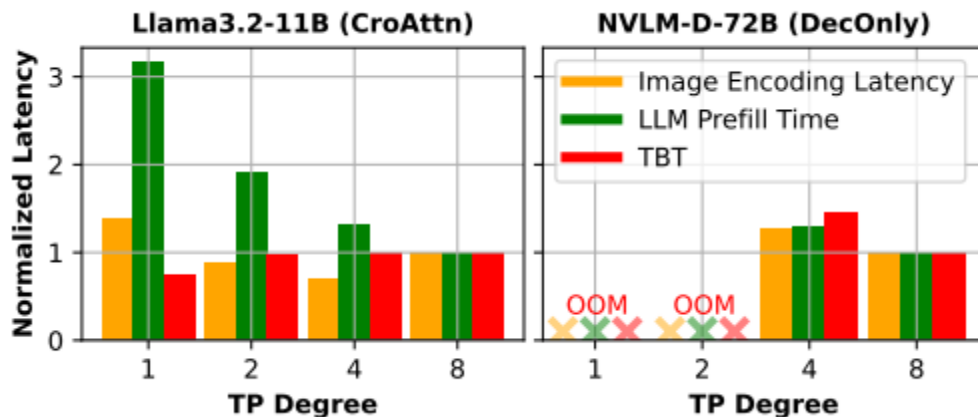
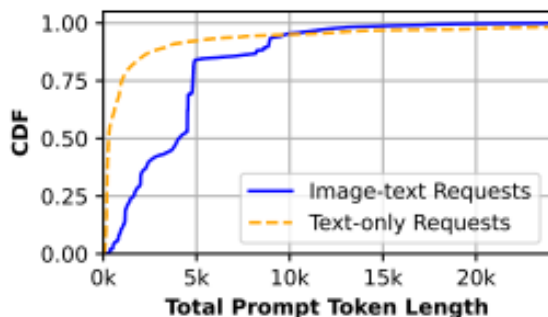


Figure 9. Impact of the tensor parallelism (TP) degree on the median latency of each stage for CroAttn-based and DecOnly LMMs. Latency is normalized to that of TP-8.

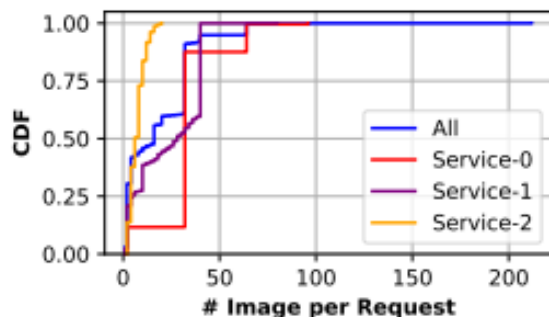
Insight 5: Treating the image encoder and LLM backend as a monolith limits parallelism flexibility and degrades performance. Decoupling them enables independent scaling and optimized efficiency through pipelining.

Production LMM Trace Analysis

Request Heterogeneity



(a) Total prompt length.



(b) #Images per request.

Figure 11. LMM input characterization in production.

Insight 6: Production LMM image traffic exhibits bursty behavior independent of traffic patterns of text requests. Serving systems must dynamically scale resources to handle modality-specific bursts efficiently.

Impact of Mixed Modality

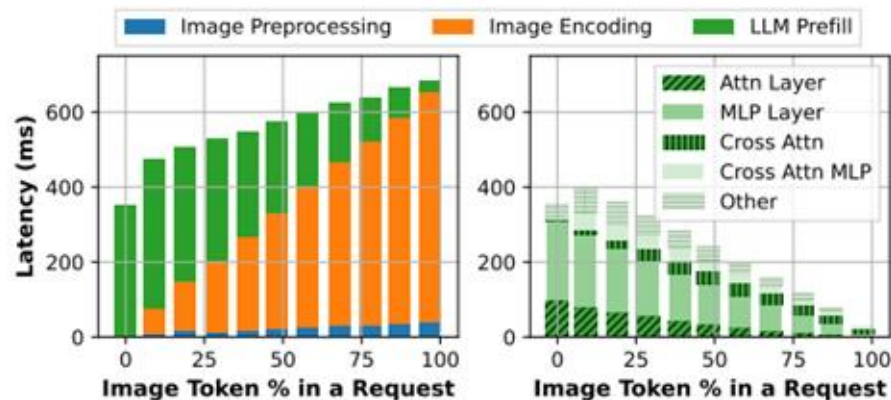


Figure 12. Llama3.2-11B (CroAttn) TTFT breakdown (left) and LLM prefill time breakdown (right) under various image-to-text token ratios in a request.

Insight 7: DecOnly models maintain consistent prefill times regardless of token modality, making total token count the key factor for request routing. In contrast, CroAttn models experience reduced prefill latency as the image token percentage increases, requiring a modality-aware routing strategy that balances both text and image token load.

ModServe - Key Design Choice

Disaggregating Image and
Text Inference

ModServe Architecture

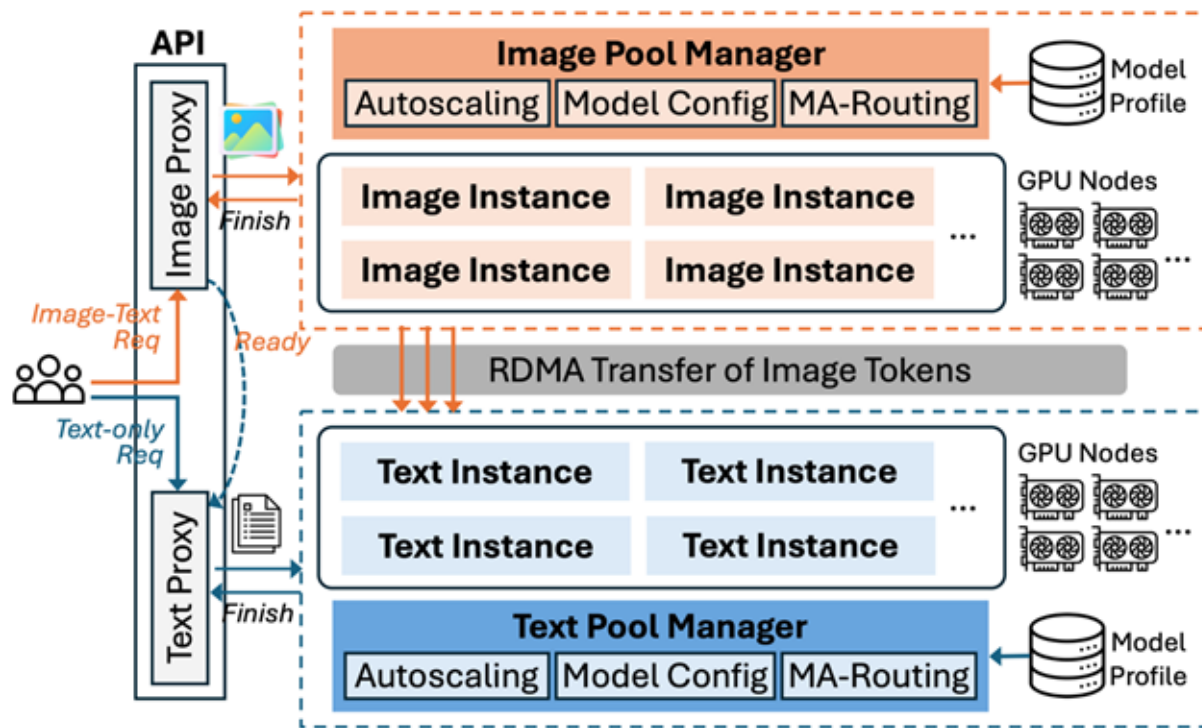
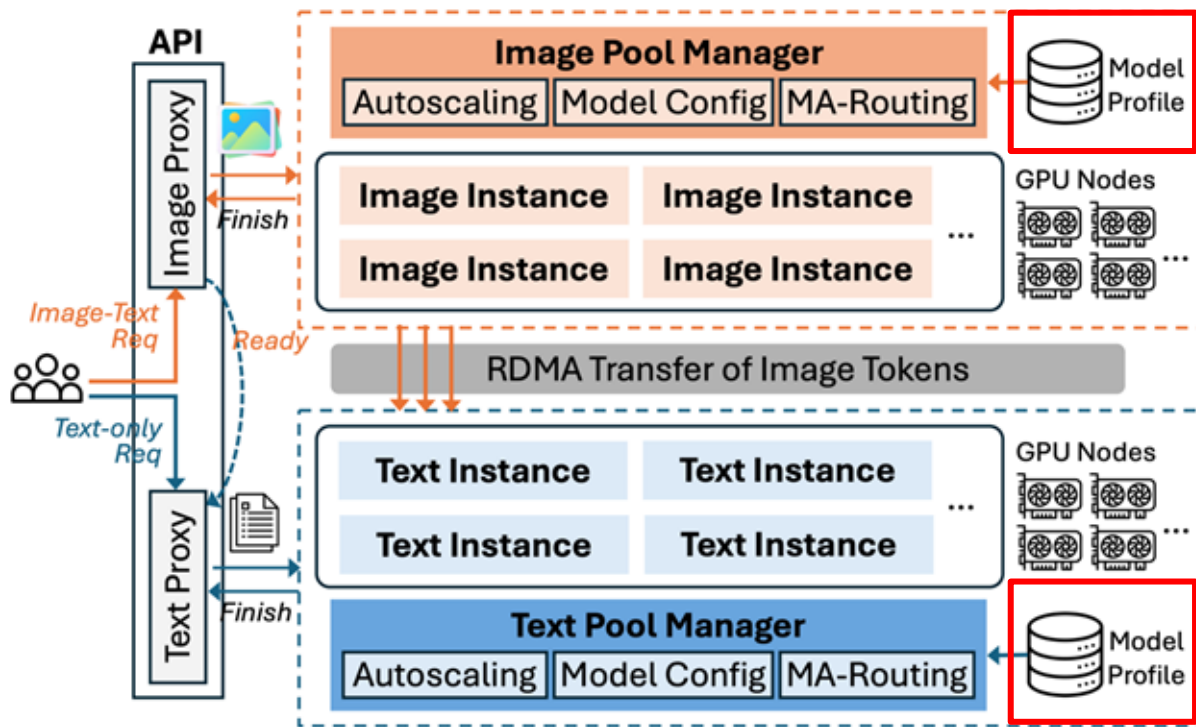


Figure 13. Overview of the MODSERVE architecture.

Offline profiler



Metrics:

Encoding
latency

Prefill
latency,
TBT

Figure 13. Overview of the MODSERVE architecture.

Separate Resource Management

Autoscaling
helps with
bursty
modality
load

Tailored
batching,
parallelism
per stage

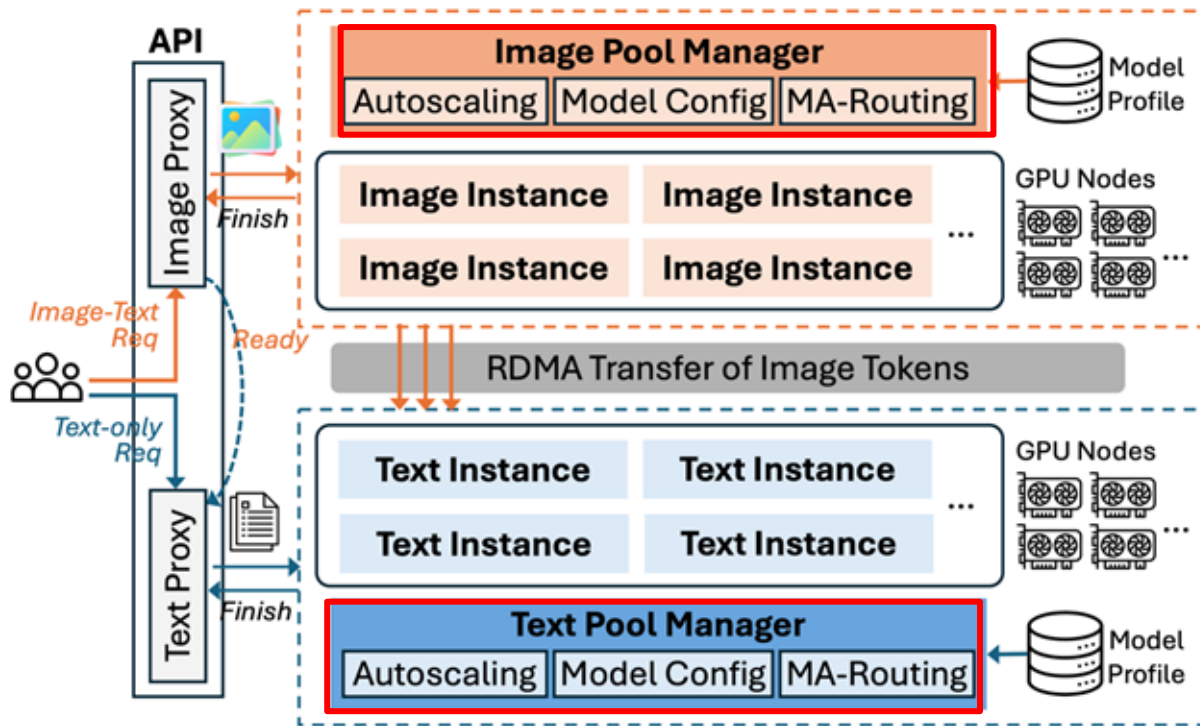


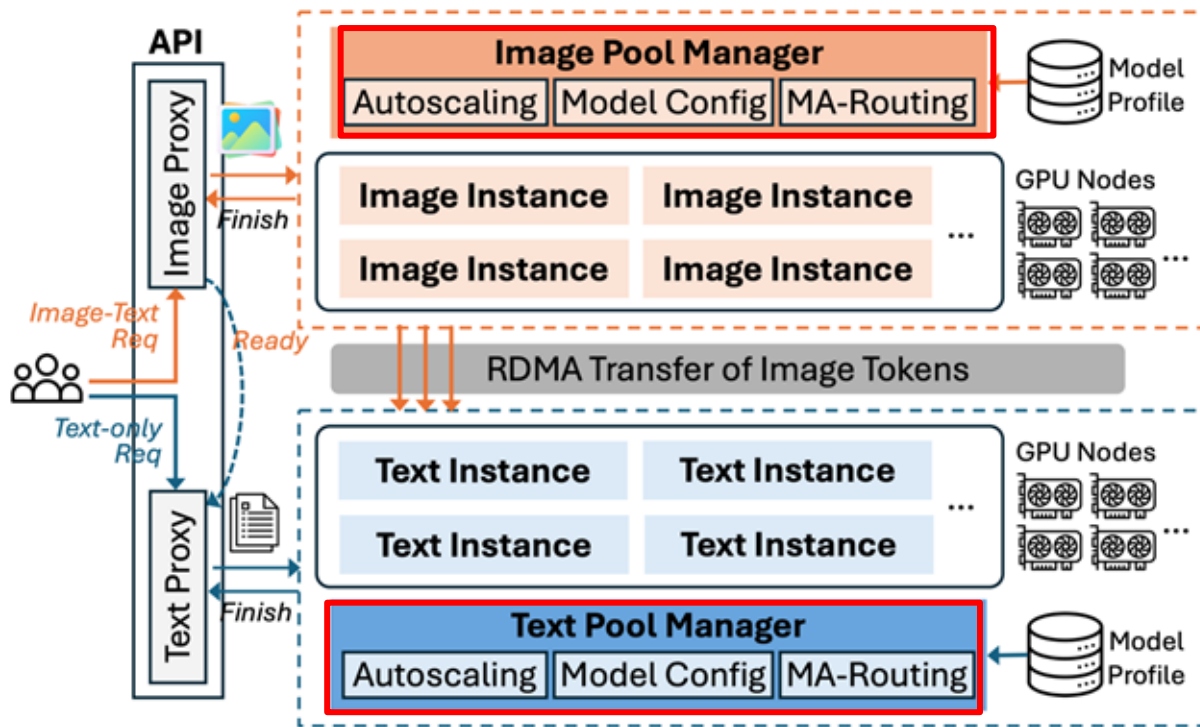
Figure 13. Overview of the MODSERVE architecture.

Online/offline behavior

Online:

$n(\text{replicas}) \sim$
ceiling(image
token
load/capacity)

num replicas
depends on
attn infra



Offline:

$N_i = \text{median}$
 $\text{QPS} * \text{encoding}$
 latency

$N_t = N_i /$
Historic
images per
request

Figure 13. Overview of the MODSERVE architecture.

Routing

Separate
proxies for
each
modality

Text-only
requests
skip image
instances

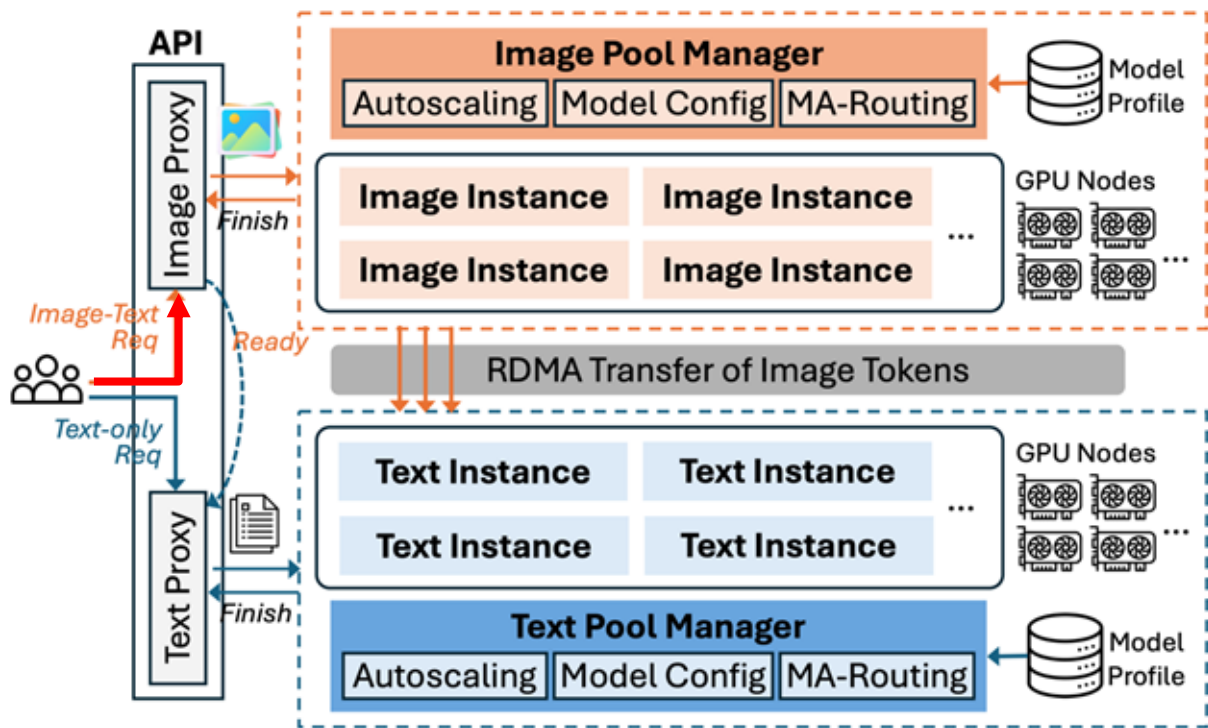


Figure 13. Overview of the MODSERVE architecture.

Routing

4. Notify proxy of completion

1. Route to image server with least tokens
2. Spread large requests across multiple instances

3. Image preprocessing on CPU

Image encoding on GPU

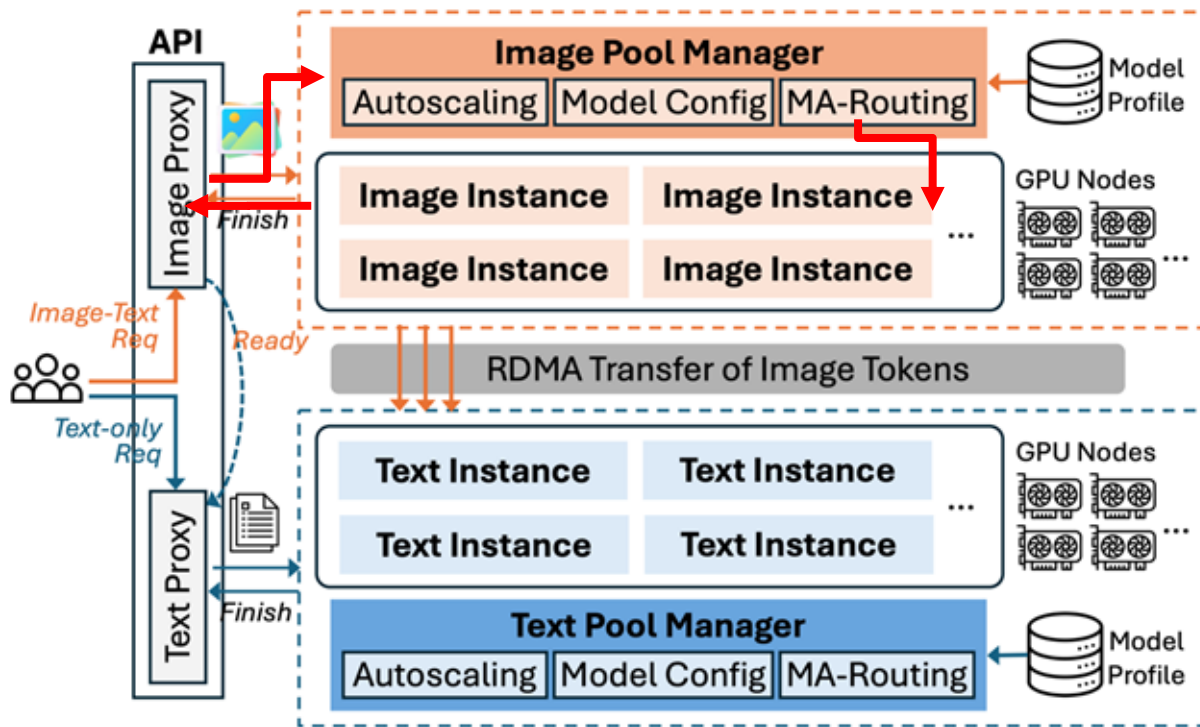
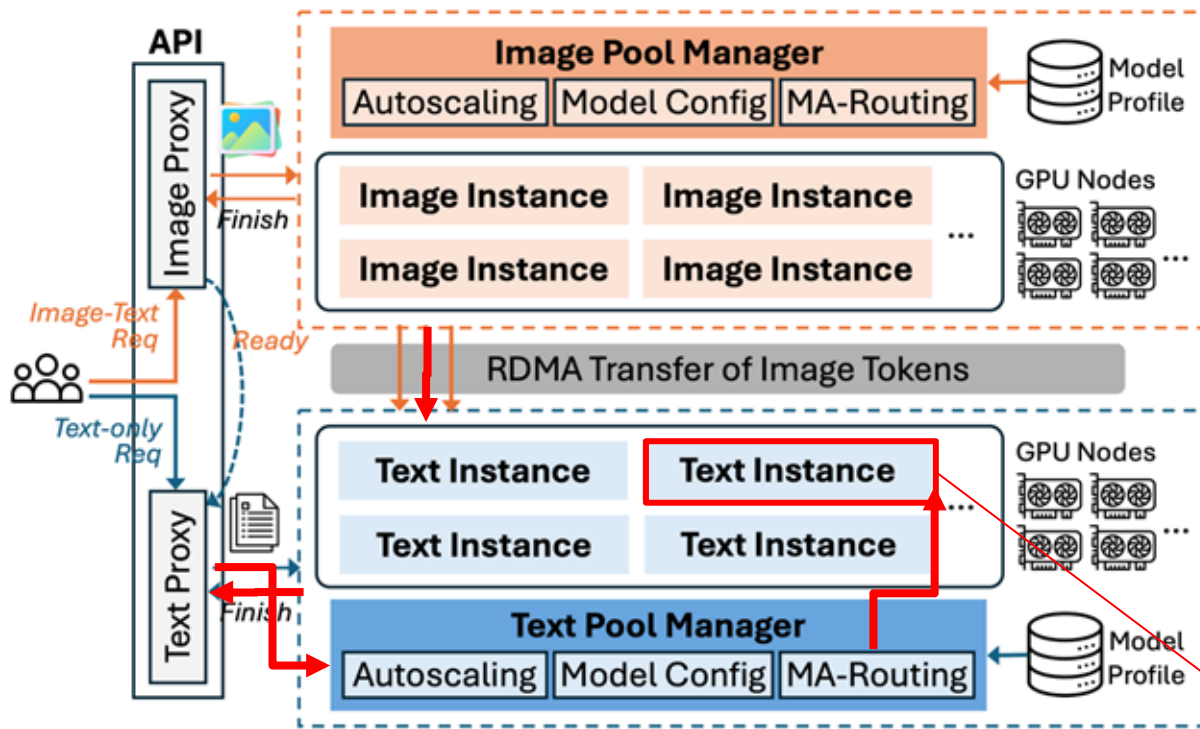


Figure 13. Overview of the MODSERVE architecture.

Control flow

2. Fetch image tokens via pull-based RDMA
(per request latency = 5ms)

1. Route to most
available
text
instance



3. Prefill
and decode

SLO-driven
request
scheduling

Figure 13. Overview of the MODSERVE architecture.

Implementation

- ~5000 lines of code
- Text instances based on vLLM
- Image instances built on HuggingFace Transformers
- Pool managers = gRPC servers

Evaluation

Test Bed

16
DGX-A100 servers
(128 GPUs)

Within Server: NVLINK
3.0

Across Servers: InfiniBand

Models

CroAttn: Llama 3.2-11B

DecOnly: InternVL26B

Datasets

- ShareGPT-4o
- Production Trace from Azure Clusters

Metrics

P99 latency against SLO
(Strict + relaxed)

Baseline systems

- vLLM
- ModServe-Decoup
(1 + decoupled instances)
- ModServe-Sched (2 + modality-aware scheduling)

And ModServe (3 + modality-aware routing)

Static Resource Allocation

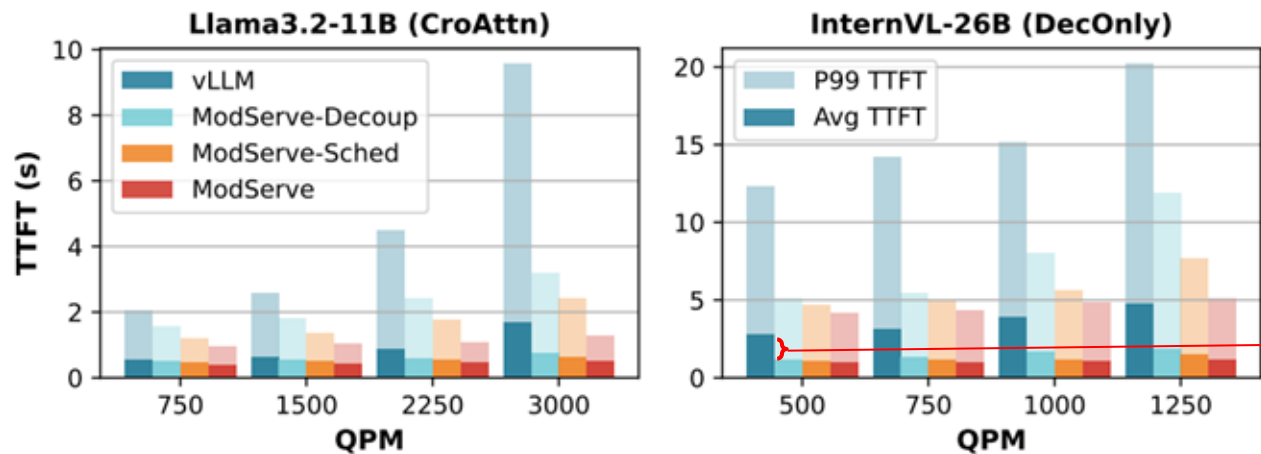


Figure 14. TTFT comparison with fixed 16 servers (128 GPUs) without autoscaling.

Less efficient prefill, larger gain from decoupling

Better throughput at different cluster sizes

Modality-aware routing =
Better throughput
scaling

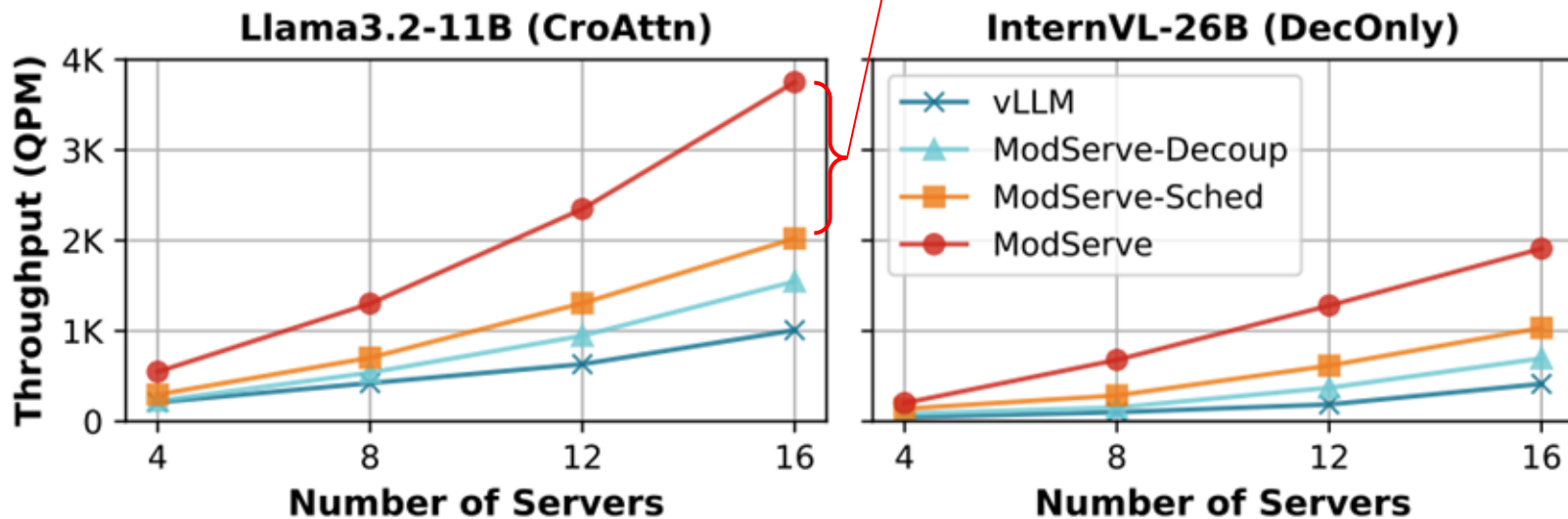


Figure 15. Maximum load meeting SLO.

Better GPU allocation
during bursty modality load

Better prefill efficiency with
CroAttn model, therefore
less GPUs needed

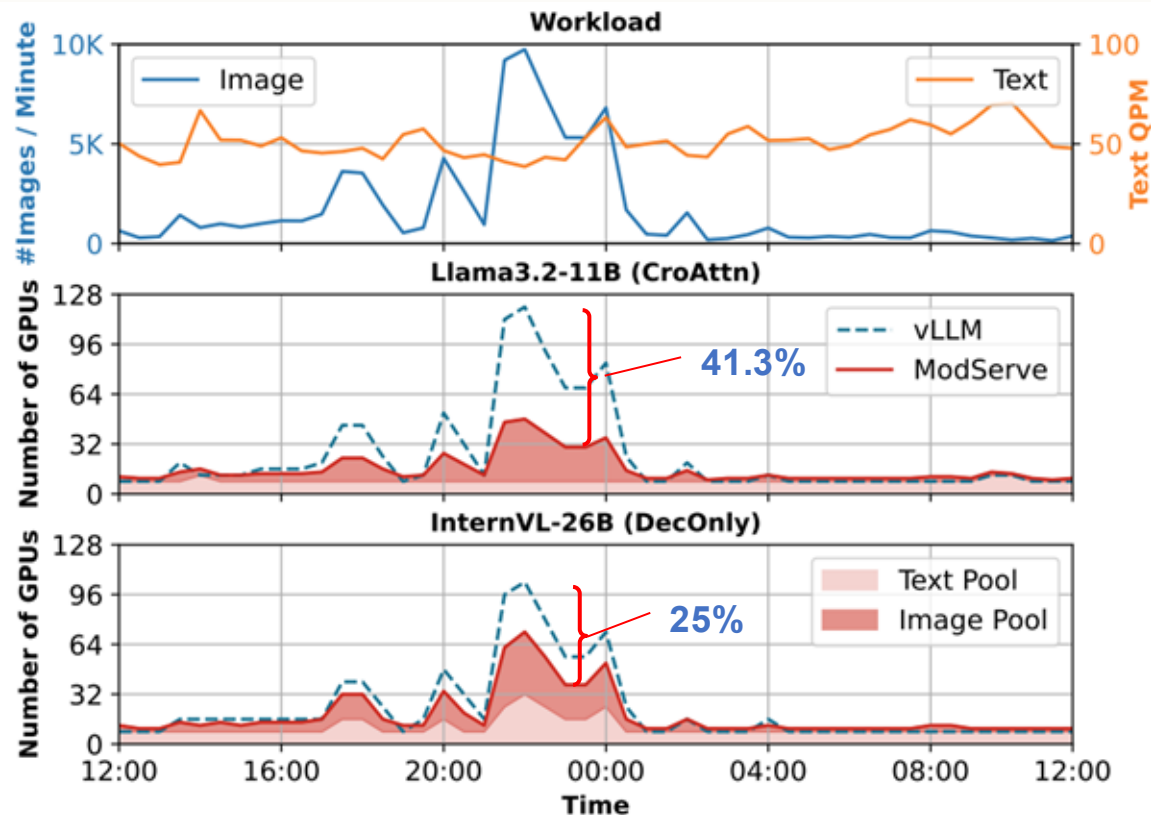


Figure 16. GPU allocation with autoscaling (up to 16 servers) during a one-day interval on the production trace^c

Sensitivity Study

Better resource
utilization under the
same SLO constraints

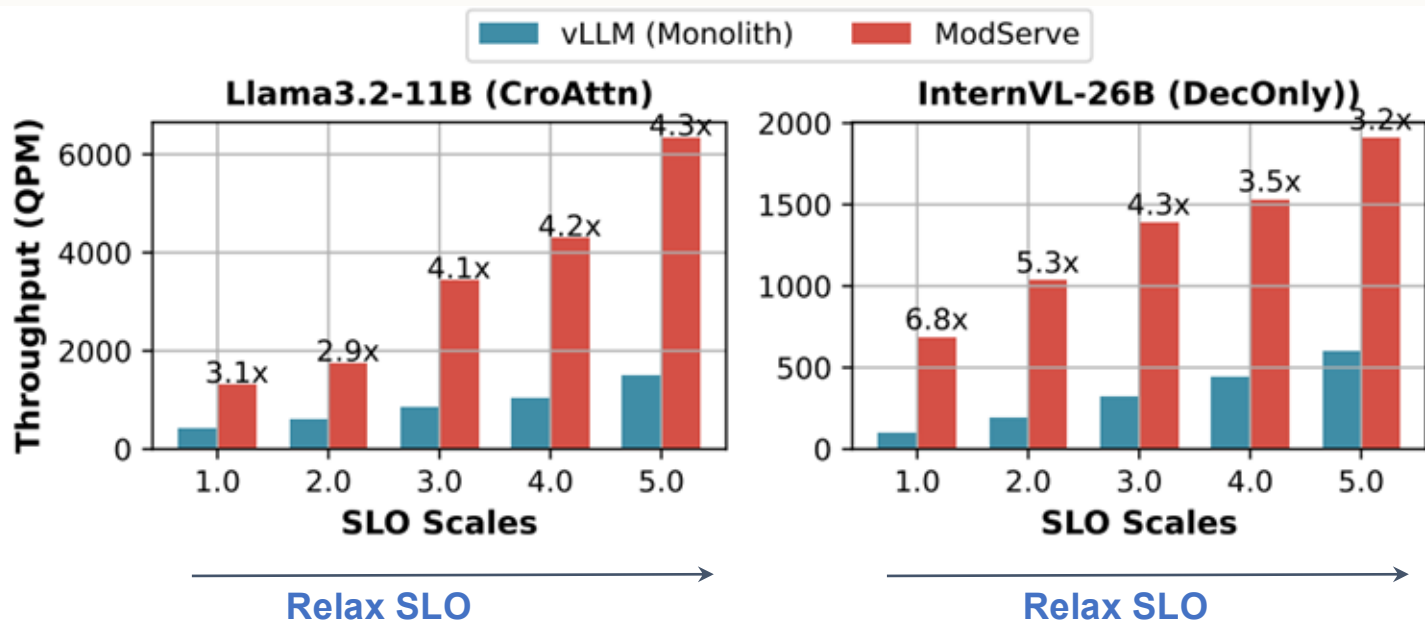


Figure 17. Throughput impact varying the SLO scale.

Image req % v/s instance allocation

- Text: TP4, Image: TP1
- As we stop over provisioning image instances, benefits from decoupling disappear
- Reduced text load as image req % increases explains non-linear TTFT in DecOnly design for image-over provisioned allocation mixes
- When Image:Text ratio > 2.4, ModServe improves on baseline by 18.4× for Llama3.2 and 9.2× for InternVL



(a) Llama3.2-11B CroAttn LMM.



(b) InternVL-26B DecOnly LMM.

Figure 18. Impact of image request percentage (Y-axis) and instance allocation (X-axis), *i.e.*, #Text Instances (TP4) : #Image Instances (TP1) on 8 servers (64 GPUs).

Strengths & Weaknesses

Strengths:

- Decoupling unlocks modality-aware optimizations
- Better resource utilization under SLO
- Handles bursty modality load through autoscaling
- Better throughput scaling

v/s

Weaknesses:

- Requires good bandwidth / colocated instances to reduce comm cost
- Fixed allocation to each modality
- Only heuristic instance allocation guidance

Comparison to existing work

Existing Characterizations:

Multi-modal generation
trace data only

LMM Serving Solutions:

Model-specific solutions
(trade off performance)

Text-Only Serving Solutions:

Applicable to text prefill
and decode

Future Work

Integrate state-of-the-art
batch scheduling
techniques

Better leader election
and fault tolerance
paradigms

Autoscaling hysteresis
prevention techniques to
prevent excessive scaling

Cornserve: Efficiently Serving Any-to-Any Multimodal Models

Authors: Ma, Chung, Ahn, Liang, Jajoo, Lee, Chowdhury
University of Michigan · USC · Cisco Research

Presenters: Pranav Premchand, Chinmay Dandekar, Ambika Sharan



What Are Any-to-Any Models?

Multimodal models accept combinations of text, image, video, and audio as input and output generalized from text-only LLMs.

Multimodal Encoders

Convert image/video/audio → token embeddings

LLMs

Reason over text + embeddings, generate text tokens

Multimodal Generators

Produce images, video, or audio from text/embeddings

Example Models

- **Input only (MLLM)**
Qwen 2.5 VL, InternVL 3, Llama 4
- **Output generation**
Qwen Image, Qwen Image Edit
- **Full Any-to-Any**
Qwen Omni, DeepSeek Janus

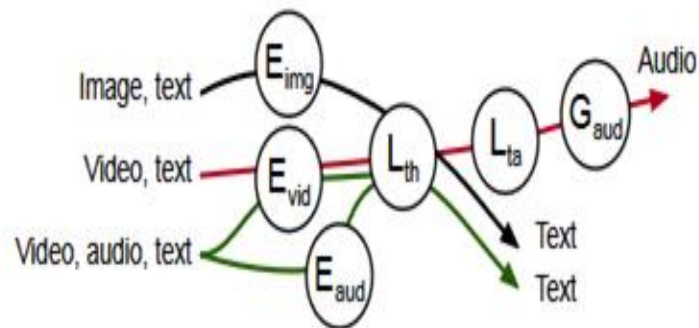
Heterogeneity in Any-to-Any Models

Request & Computation Path

Each request invokes a different subset of model components

Azure traces: 6%–40% of requests are text-only (skip encoder entirely)

Components receive different request rates → load imbalance



(b) *Multimodal input, multimodal output*

Heterogeneity in Any-to-Any Models (cont.)

	Image input	Video input	Audio input	Text in/out	Audio output	Image output
Qwen 2.5 Omni	15.64	1.28	34.04	1.09	0.28	
Qwen 2.5 VL	12.04	0.89		0.32		
InternVL 3	1.13	0.74		0.28		
Qwen-Image				15.67		0.20

Maximum throughput (req/s) of modality components
on one A100 GPU

Existing Systems Are Point Solutions

Gap 1 — Limited Model Support

- ✗ vLLM / SGLang: text-only & multimodal input LLMs
- ✗ xDiT / HF Diffusers: image/video generation only
- ✗ No system handles multimodal input AND output together

Existing Systems Are Point Solutions

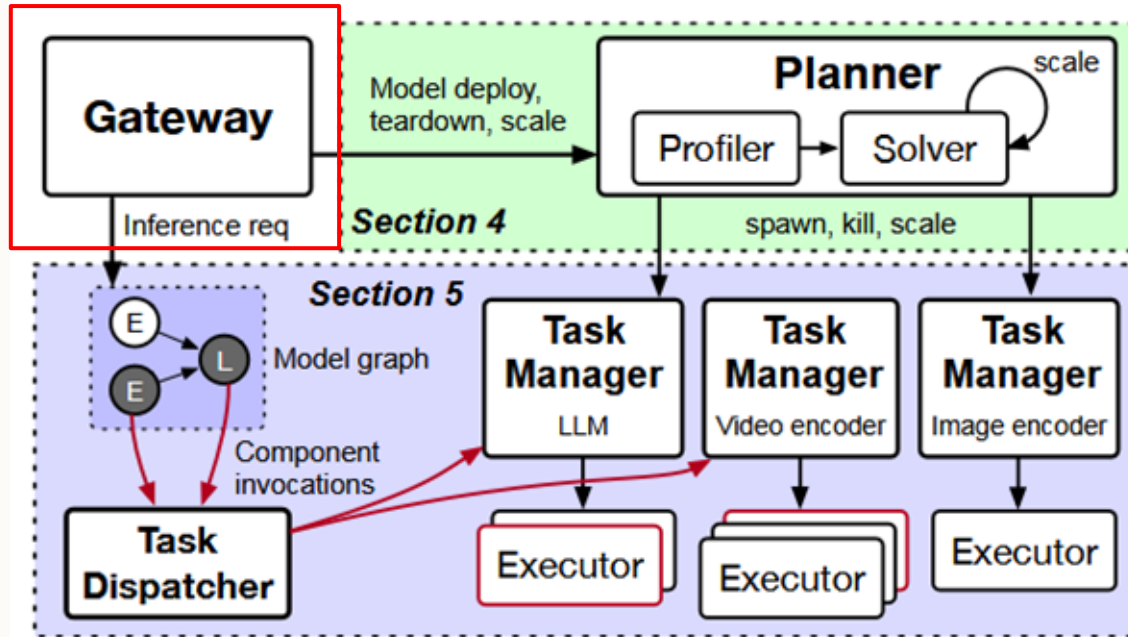
Gap 1 — Limited Model Support

- ✗ vLLM / SGLang: text-only & multimodal input LLMs
- ✗ xDiT / HF Diffusers: image/video generation only
- ✗ No system handles multimodal input AND output together

Gap 2 — Opinionated Deployment

- ✗ PD disaggregation prescribed for text-only LLMs
- ✗ EPD disaggregation prescribed for MLLMs
- ✗ No workload-adaptive hybrid approach possible

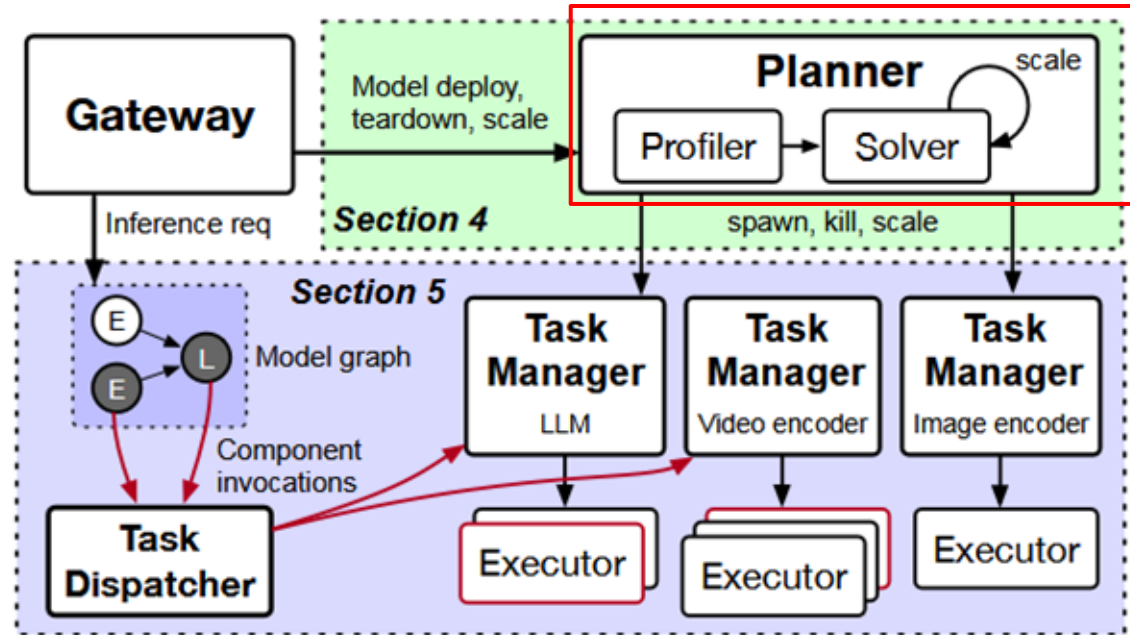
Cornserve Architecture



Gateway

Entry point for model deployment commands and inference requests from clients.

Cornserve Architecture

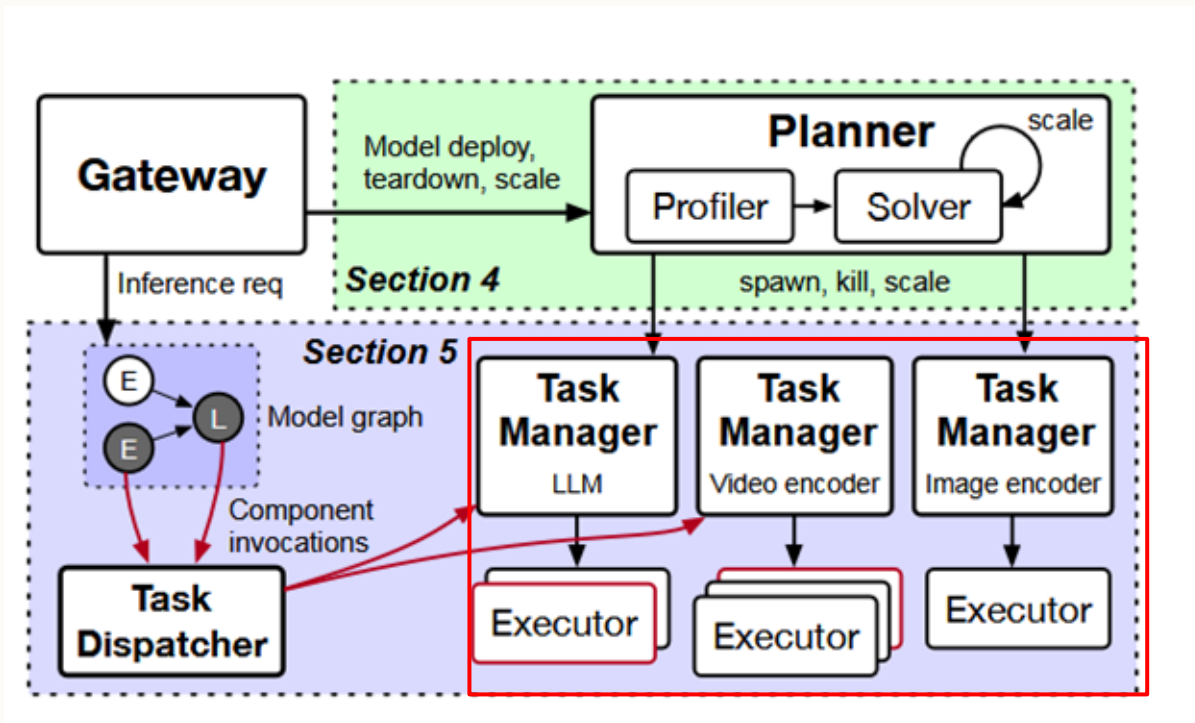


Planner

Allocates resources; includes Profiler + Solver.

Determines optimal deployment plan.

Cornserve Architecture



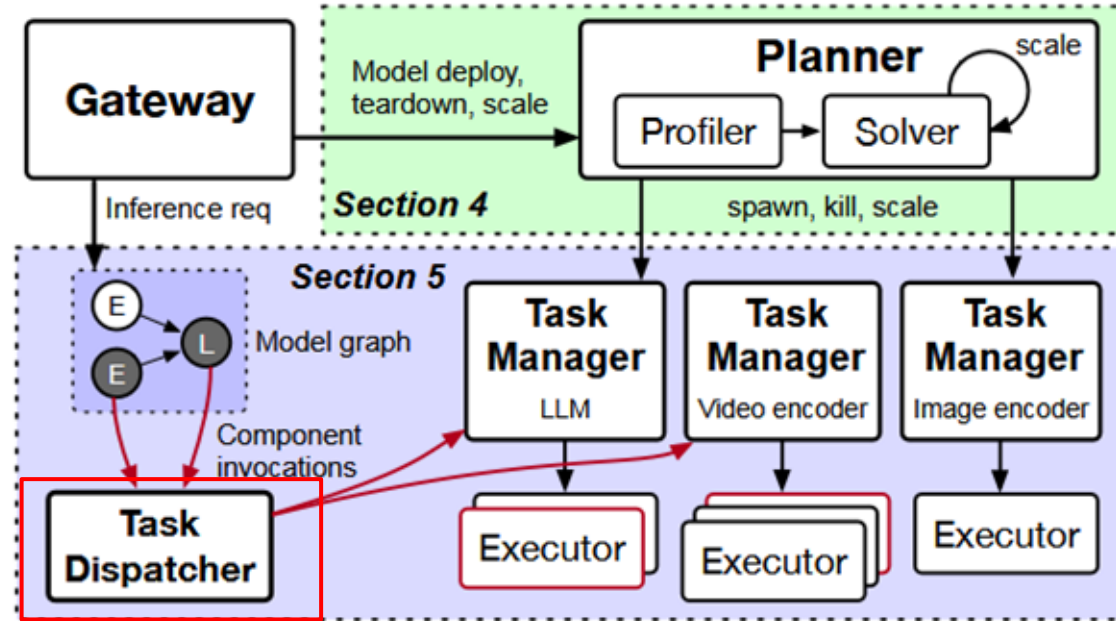
Task Manager

Manages a set of Task Executor replicas for one deployment option.

Task Executor

Runs model components (encoder, LLM, generator) on GPUs.

Cornserve Architecture



Task Dispatcher

Dispatches component invocations to the correct Task Manager / Executor.

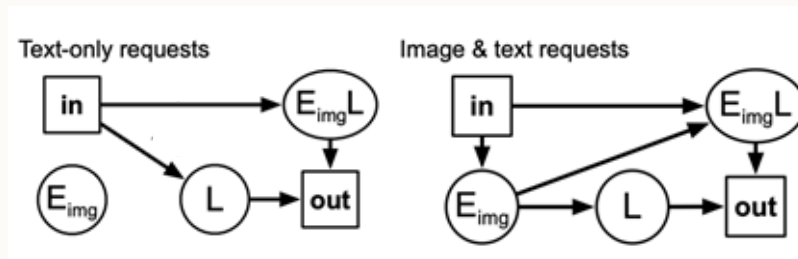
Planner Inputs

Model Definition

Any-to-Any model executor
graph with valid components and
invocation paths.

Workload Distribution

Representative request dataset
decomposed into disjoint request
types.



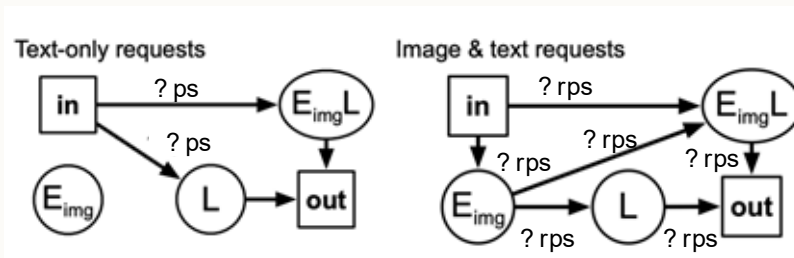
Workload (request distribution)

Text-only : Image & text = 2 : 9

Planner Outputs

Request Splitting

Split request types across
executor paths



Replica Allocation

Determine number of replicas for
each executor

	Work demand	Replicas (round up) GPUs	
E_{img}		?	?
L		?	?
$E_{img}L$		?	?
		=?	

Planning Goals

Different workloads require different deployment configurations.



1

Maximize Throughput

Maximize request throughput given a GPU budget

2

Minimize GPU Usage

Minimize GPU usage given target request throughput

Planning Goals

Different workloads require different deployment configurations.

1

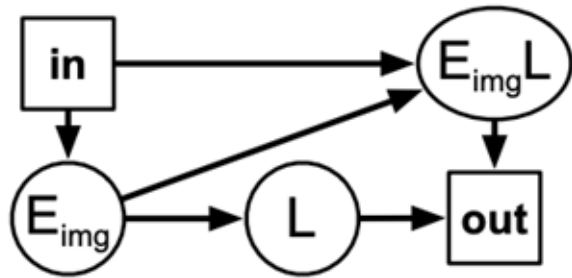
Maximize Throughput

Maximize request throughput given a GPU budget

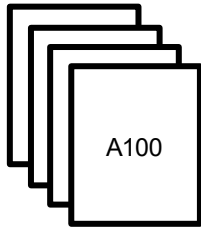
2

Minimize GPU Usage

Minimize GPU usage given target request throughput



Instance of full model running on
4 GPUs



Definition of 1 Cell

Cell-Based Planning

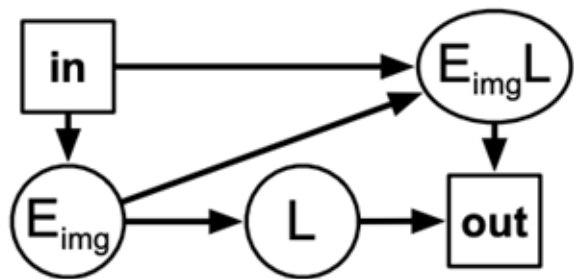
Cell Definition

Fixed GPU allocation

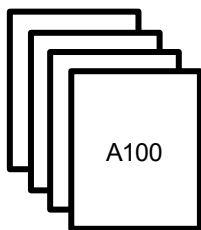
running complete Any-to-

Any model instance. Size

always power of two.



Instance of full model running on
4 GPUs



Definition of 1 Cell

Cell-Based Planning

Cell Definition

Fixed GPU allocation
running complete Any-to-
Any model instance. Size
always power of two.



Reduced Complexity

Reduces problem hardness from
GPU count to constant executor
count.

Planner Steps

1

Profile Executor

2

Maximize Throughput for 1 Cell

3

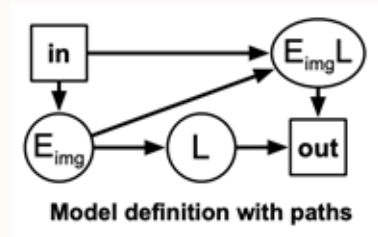
Maximize Throughput across Cells

Planner Steps

1

Profile Executor

Compute the maximum throughput a single replica of each executor can sustain, and how many GPUs are needed per replica



Throughput per request type GPUs per replica

	Image	Text	Image & text	
E _{img}	10.0			1
L		3.33		2
E _{img} L		3.03	2.27	2

Profiling results

Planner Steps

2

Maximize Throughput for 1 Cell

Use a Mixed Integer Linear Program to maximize total throughput given a Budget of GPUs

$$\begin{aligned} \max_{x,r,R} \quad & R \\ \text{s.t.} \quad & \sum_{p \in \mathcal{P}_t} x_{t,p} = \pi_t R \quad \forall t \in T, \end{aligned} \tag{13}$$

$$\sum_t \sum_{p \in \mathcal{P}_t} x_{t,p} / w_{s,t,p} \leq r_s \quad \forall s \in S, \tag{14}$$

$$\sum_{s \in S} c_s r_s \leq B, \tag{15}$$

$$x_{t,p} \geq 0, \quad r_s \in \mathbb{Z}_{\geq 0}, \quad R \geq 0.$$

Planner Steps

2

Outputs

$$\max_{x,r,R} R \text{ (Throughput)}$$

$$\text{s.t. } \sum_{p \in \mathcal{P}_t} x_{t,p} = \pi_t R \quad \forall t \in T, \quad (13)$$

$$\sum_t \sum_{p \in \mathcal{P}_t} x_{t,p} / w_{s,t,p} \leq r_s \quad \forall s \in S, \quad (14)$$

$$\sum_{s \in S} c_s r_s \leq B, \quad (B = \text{GPU Budget}) \quad (15)$$

$$x_{t,p} \geq 0, \quad r_s \in \mathbb{Z}_{\geq 0}, \quad R \geq 0.$$

Text-only requests

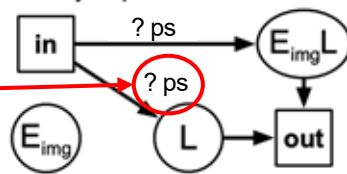
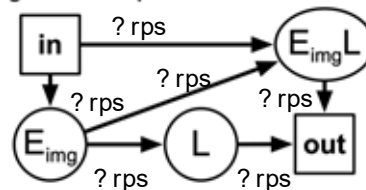


Image & text requests



	Work demand	Replicas (round up)	GPUs
E_{img}		?	?
L		?	?
$E_{img} L$		?	?
			=?

Planner Steps

2

Constants

$$\max_{x,r,R} R \text{ (Throughput)}$$

$$\text{s.t.} \quad \sum_{p \in \mathcal{P}_t} x_{t,p} = \pi_t R \quad \forall t \in \mathcal{T}, \quad (13)$$

$$\sum_t \sum_{p \in \mathcal{P}_t} x_{t,p} / w_{s,t,p} < r_s \quad \forall s \in \mathcal{S}, \quad (14)$$

$$\sum_{s \in \mathcal{S}} c_s t_s \leq B, \quad (15)$$

$$x_{t,p} \geq 0, \quad r_s \in \mathbb{Z}_{\geq 0}, \quad R \geq 0.$$

Workload (request distribution)

Text-only : Image & text = 2 9

	Throughput per request type			GPUs per replica
	Image	Text	Image & text	
E _{img}	10.0			1
L		3.33		2
E _{img} L		3.03	2.27	2

Profiling results

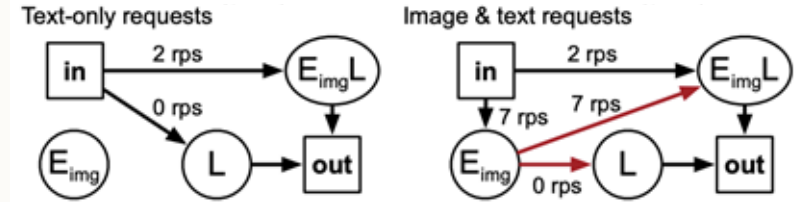
Planner Steps

2

Maximize Throughput for 1 Cell

Use a Mixed Integer Linear Program to maximize total throughput given a Budget of GPUs

GPU Budget: 8,
Output:



	Work demand	Replicas (round up) GPUs	
E_img	$7 / 10.0 = 0.70$	4	4
L	0	0	0
E_img L	$(2 + 7) / 3.03 + 2 / 2.27 = 3.85$	2	4

Total Throughput: 11rps

Planner Steps

3

Maximize Throughput across Cells

Filter through inefficient cells and remove them

Input: Maximum cell exponent $c_{\max}$ (max size $2^{c_{\max}}$)

Output: Efficient cells $\mathbf{C} = \{(g[c], R[c])\}$

▷ Multicommodity network design (§4.2)

1 **for** $c \leftarrow 0$ **to** $c_{\max}$ **do**

2 $g[c] \leftarrow 2^c$

3 $R[c] \leftarrow \text{BudgetMNDP}(g[c])$

▷ Filter inefficient cells

4 $\mathbf{C} \leftarrow \{(g_0, R_0)\}$

5 $c^* \leftarrow 0$ ▷ Index of largest efficient cell so far

6 **for** $c \leftarrow 1$ **to** $c_{\max}$ **do**

7 **if** $R_c > 2^{c-c^*} R_{c^*}$ **then**

8 $\mathbf{C} \leftarrow \mathbf{C} \cup \{(g_c, R_c)\}$

9 $c^* \leftarrow c$

10 **return** $\mathbf{C}$

Algorithm 1: Planning efficient power-of-two cells.

Planning Goals

Different workloads require different deployment configurations. The planner finds the optimal mixture of executor deployment options and replica counts.

1

Maximize Throughput

Maximize request throughput given a GPU budget

2

Minimize GPU Usage

Minimize GPU usage given target request throughput



Planning Goals

Different workloads require different deployment configurations. The planner finds the optimal mixture of executor deployment options and replica counts.

1

Maximize Throughput

Maximize request throughput given a GPU budget

2

Minimize GPU Usage

Minimize GPU usage given target request throughput

Planner Steps

4

Minimize GPUs given Throughput Requirement

Use a greedy algorithm (inverse of knapsack) to minimize GPUs

Input: Efficient cells $\mathbf{C} = \{(g_c, R_c)\}$

Target request throughput R^*

Output: Cell mixture $M : c \mapsto \text{count}$

Achieved throughput R and GPUs used N

```
1  $R \leftarrow 0, N \leftarrow 0, M \leftarrow \{\}$ 
2 while  $R < R^*$  do
     $\triangleright$  Largest cell that fits in remaining throughput
3    $g^* \leftarrow \max\{g_c : R_c \leq R^* - R, (g_c, R_c) \in \mathbf{C}\}$ 
4   if  $g^* = \emptyset$  then
5      $\lfloor g^* \leftarrow \min\{g_c : (g_c, R_c) \in \mathbf{C}\}$   $\triangleright$  Last cell
6    $R \leftarrow R + R_{g^*}, N \leftarrow N + g^*, M[g^*] \leftarrow M[g^*] + 1$ 
7 return  $(M, R, N)$ 
```

Algorithm 2: Finding the optimal cell mixture that meets target throughput with minimal GPUs.

Planning Goals

Different workloads require different deployment configurations. The planner finds the optimal mixture of executor deployment options and replica counts.

1

Maximize Throughput

Maximize request throughput given a GPU budget

2

Minimize GPU Usage

Minimize GPU usage given target request throughput



Load Adaptation

1

Load Changes

Request load varies over time

2

Quick Recalculation

Run **only** Planner Stage 3 again

3

Efficient Update

Keep identical cells, spawn or kill differences

Distributed Runtime

Planner decides **how many GPUs + where**

Runtime decides **how to execute requests** using that plan. It must support the following

- (1) APIs/abstractions that allow model developers to express Any-to-Any model computations conveniently
- (2) a mechanism that supports flexible Any-to-Any model control flow execution on top of that abstraction
- (3) a runtime stack that routes and load balances model component executions across executors as planned

Record & Replay Execution

Cornserve provides a library of unit task classes that express atomic model components, and model developers compose unit tasks within a composite task class

Step 1: **Record phase**

- execute with mock outputs
- capture task graph + dependencies
- **The record pass is instantaneous, as no actual model computation is performed.**

Step 2: **Replay phase**

- Invoke composite task again in replay mode. Unit tasks now fetch real results from executors (GPUs)
- Task Dispatcher:
 - a. routes tasks to correct executors
 - b. executes real computation
- Replay uses real outputs to:
 - a. follow same control flow
 - b. assemble final response

Request-Static Routing

Problem: Multiple executors → need load-balanced routing

Solution: Planner already computes rate split across paths

Runtime Steps

1. Identify **request type** (e.g., text-only, image+text)
2. **Sample path** using planner probabilities
3. Route request **statically along that path**

Record builds the logical computation graph for a request, the planner assigns probabilities to different execution paths for that graph, and static routing selects one of these paths at the start of execution using those probabilities—after which replay executes the graph along that fixed path without any runtime adjustments.

Implementation

System Setup

- Built on **Kubernetes** → manages deployment, scaling, and distributed services
- Uses **vLLM (v0.9.2)** as LLM executor

Intra-node (same machine)

- Use **shared memory (/dev/shm)**
- Avoids contention with NVLink/NVSwitch

Inter-node (different machines)

- Use **RDMA via UCX**
- Enables low-latency, high-throughput communication

Metrics:

- Throughput (req/s)
- Latency (P50, P90, P99)

Setup: 2 nodes × 8 A100 GPUs

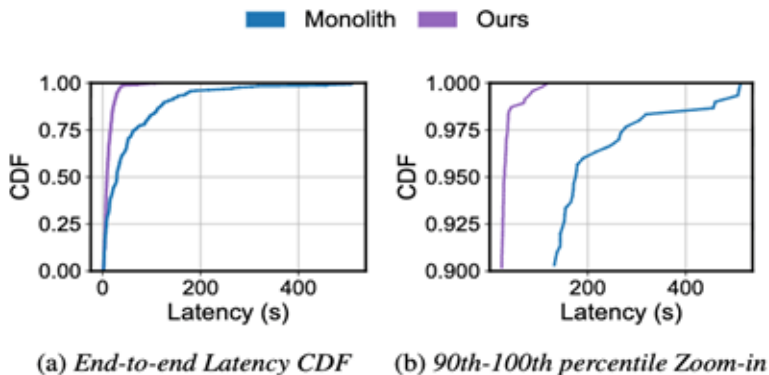
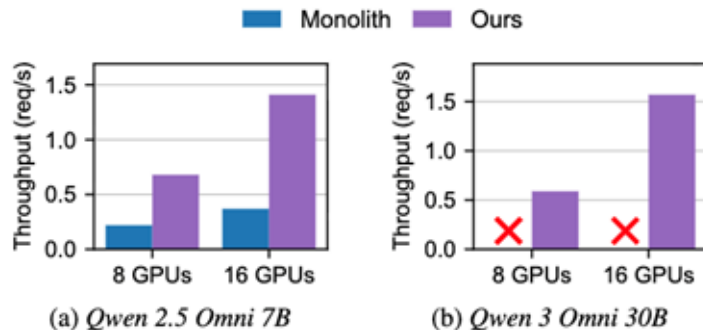
Text and audio output models

Qwen 3 Omni 30B and Qwen 2.5 Omni 7B

Throughput: Up to **3.81× throughput improvement**

- Handles models that:
 - fail under monolithic (OOM)
- Balances heterogeneous components:
 - LLM vs generator scaling

Latency: Up to **5.79× P99 latency reduction**



Text-only Output Models

MLLMs: Qwen 2.5 VL 32B and InternVL 3 38B

For text-only output and image output models, Cornserve **can match or outperform** the best existing deployment strategies for each model

Why:

- Disaggregating encoders improves component balance
- But **reduces GPUs available for LLM**
- Cornserve's planner detects this
- Chooses **monolithic deployment**

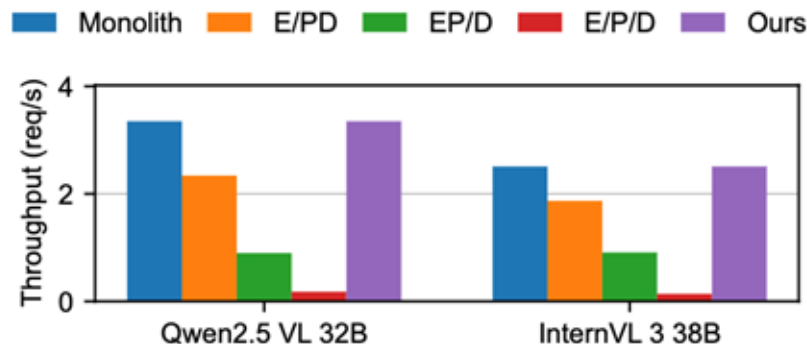


Image-only Output Models

Setup

- Task: **text** → **image generation**
- Pipeline:
 - LLM (Qwen 2.5 VL 7B) → embeddings
 - **DiT (20B diffusion model)** → image generation

Key Result

- Cornserve **chooses monolithic deployment**
- Matches monolithic throughput

This is because the drastic computation heterogeneity between the LLM component and the DiT component cannot be balanced within a 16-GPU cell.

Static vs Dynamic Routing

Setup

- InternVL 3 38B on 16 GPUs
- Mixed deployment:
 - 4 dedicated encoders
 - 6 monolithic executors
- 67.5% requests routed via encoders

Results

- Static routing: **9.27 req/s**
- Dynamic routing: **2.50 req/s**
- **3.7× higher throughput**

Conclusion

First system for **Any-to-Any multimodal serving**

Introduces a **planner-driven deployment**

Selects **optimal strategy per model + workload**

Performance Insight

- Matches best systems for:
 - **LLM-heavy (decode-dominant) workloads**
- Outperforms for:
 - **complex multimodal pipelines**
- Improves:
 - throughput
 - latency

Strengths & Weaknesses

Strengths:

- First framework that explicitly supports Any-to-Any Model
- Highly adaptable to serving techniques
- Optimized for workload distribution

v/s

Weaknesses:

- Is network-topology unaware
- Non-deterministic outputs are not accounted for
- Relies on user to provide accurate and static workload-distribution

Future Work

Optimizing for SLOs or
goodput rather than raw
throughput

Real time adaptation to
workload distribution
changes

Finding a network-
topology aware optimal
configuration